

Cursor IDE 進階使用手冊

高級功能 | 自訂設定 | 工作流優化

目錄

1. [進階介面自訂](#)
2. [深度 AI 功能配置](#)
3. [高級開發工作流](#)
4. [性能優化與調試](#)
5. [進階版本控制](#)
6. [團隊協作功能](#)
7. [自動化與擴展開發](#)
8. [安全與隱私設定](#)
9. [進階快捷鍵自訂](#)
10. [故障排查與優化](#)

進階介面自訂

設定檔案 (settings.json) 深度自訂

Cursor 的所有設定都儲存在 `settings.json` 中。要編輯它：

開啟設定檔案

按 `Ctrl + Shift + P` (Windows/Linux) 或 `Cmd + Shift + P` (Mac)
搜尋 "Preferences: Open Settings (JSON)"

常用高級設定

1. 編輯器基礎設定

```
{
  // 編輯器字體和外觀
  "editor.fontFamily": "Fira Code, Consolas, 'Courier New'",
  "editor.fontSize": 14,
  "editor.fontLigatures": true,
  "editor.lineHeight": 1.6,
  "editor.wordWrap": "on",
  "editor.renderWhitespace": "selection",

  // 縮進和空格
  "editor.tabSize": 2,
  "editor.insertSpaces": true,
```

```
"editor.trimAutoWhitespace": true,

// 編輯器行為
"editor.autoClosingBrackets": "always",
"editor.autoClosingQuotes": "always",
"editor.autoSurround": "languageDefined",
"editor.formatOnSave": true,
"editor.formatOnPaste": true,

// 代碼摺疊
"editor.folding": true,
"editor.foldingStrategy": "auto",
"editor.showFoldingControls": "mouseover"
}
```

2. AI 和補全設定

```
{
  // AI 補全行為
  "editor.inlineSuggest.enabled": true,
  "editor.inlineSuggest.suppressSuggestions": false,
  "editor.inlineSuggest.fontFamily": "Consolas",

  // 自動補全
  "editor.quickSuggestions": {
    "other": "on",
    "comments": "off",
    "strings": "off"
  },

  // 建議延遲（毫秒） - 數字越大延遲越久
  "editor.quickSuggestionsDelay": 10,

  // Cursor AI 特定設定
  "cursor.mode": "fast", // 或 "balanced" / "thorough"
  "cursor.contextLength": "default", // "short" / "default" / "extended"
  "cursor.enableAutocompletion": true
}
```

3. 檔案和搜尋設定

```
{
  // 排除檔案和資料夾
  "files.exclude": {
    "**/.git": true,
    "**/.svn": true,
    "**/.hg": true,
    "**/CVS": true,
    "**/.DS_Store": true,
  }
}
```

```
    "**/node_modules": true,
    "**/__pycache__": true,
    "**/*.pyc": true,
    "dist": true,
    "build": true,
    ".next": true
  },

  // 搜尋排除
  "search.exclude": {
    "**/.git": true,
    "**/node_modules": true,
    "**/dist": true,
    "**/.venv": true
  },

  // 檔案自動保存
  "files.autoSave": "afterDelay",
  "files.autoSaveDelay": 1000,
  "files.defaultLanguage": "plaintext"
}
```

4. 終端機設定

```
{
  // 默認終端外殼
  "terminal.integrated.defaultProfile.windows": "PowerShell",
  "terminal.integrated.defaultProfile.osx": "zsh",
  "terminal.integrated.defaultProfile.linux": "bash",

  // 終端機字體和大小
  "terminal.integrated.fontSize": 13,
  "terminal.integrated.fontFamily": "Consolas",
  "terminal.integrated.lineHeight": 1.4,

  // 終端機行為
  "terminal.integrated.confirmOnExit": "hasChildProcesses",
  "terminal.integrated.enableMultiLinePasteWarning": false,
  "terminal.integrated.scrollback": 5000
}
```

5. Git 和版本控制

```
{
  // Git 路徑 (如果 Git 不在系統路徑中)
  "git.path": "C:\\Program Files\\Git\\bin\\git.exe",

  // Git 自動 Fetch
  "git.autorefresh": true,
}
```

```
"git.autofetch": true,
"git.autoFetchPeriod": 180,

// 提交簽署 (GPG)
"git.enableCommitSigning": false,

// 默認提交訊息
"git.defaultCloneDirectory": "${userHome}/projects"
}
```

6. 檔案和代碼格式化

```
{
  // 默認格式化器
  "[javascript]": {
    "editor.defaultFormatter": "esbenp.prettier-vscode",
    "editor.formatOnSave": true
  },
  "[typescript]": {
    "editor.defaultFormatter": "esbenp.prettier-vscode",
    "editor.formatOnSave": true
  },
  "[json]": {
    "editor.defaultFormatter": "esbenp.prettier-vscode",
    "editor.formatOnSave": true
  },
  "[python]": {
    "editor.defaultFormatter": "ms-python.python",
    "editor.formatOnSave": true
  },
  "[html]": {
    "editor.defaultFormatter": "esbenp.prettier-vscode"
  },
  "[css]": {
    "editor.defaultFormatter": "esbenp.prettier-vscode"
  }
}
```

工作區設定 (Workspace Settings)

不同專案可以有不同設定：

1. 在專案根目錄建立 `.vscode/settings.json`
2. 在此檔案中覆蓋全域設定

範例：特定專案的 Python 配置

```
{
  "[python]": {
    "editor.defaultFormatter": "ms-python.python",
    "editor.formatOnSave": true,
    "editor.rulers": [80, 120],
    "editor.tabSize": 4
  },
  "python.linting.enabled": true,
  "python.linting.pylintEnabled": true,
  "python.formatting.provider": "black"
}
```

🎨 主題和色彩自訂

建立自訂主題

1. 建立 `~/.config/Cursor/themes/` 目錄
2. 建立 `my-theme.json`

範例主題檔案

```
{
  "name": "My Custom Dark Theme",
  "type": "dark",
  "colors": {
    "editor.background": "#1e1e1e",
    "editor.foreground": "#d4d4d4",
    "editor.lineNumbersBackground": "#1e1e1e",
    "editor.lineNumber": "#858585",
    "editor.selectionBackground": "#264f78",
    "editor.findMatchBackground": "#9e6a03",
    "editorBracketMatch.border": "#888888",
    "editorBracketMatch.background": "#0064001a",
    "editor.wordHighlightBackground": "#575757b3"
  },
  "tokenColors": [
    {
      "name": "Comment",
      "scope": "comment",
      "settings": {
        "foreground": "#6a9955"
      }
    },
    {
      "name": "String",
      "scope": "string",
      "settings": {
        "foreground": "#ce9178"
      }
    }
  ]
}
```

```
    },  
    {  
      "name": "Keyword",  
      "scope": "keyword",  
      "settings": {  
        "foreground": "#569cd6"  
      }  
    }  
  ]  
}
```

應用自訂主題

Ctrl + Shift + P → "Color Theme" → 選擇您的自訂主題

深度 AI 功能配置

AI 模型選擇與配置

1. 模型選擇設定

開啟 AI 設定

Ctrl + Shift + P → "Preferences: Open AI Settings"

配置不同的 AI 模型

```
{  
  // 選擇 AI 模型供應商  
  "cursor.aiModel": "gpt-4-turbo", // 或 "claude-3-opus" / "gpt-4"  
  
  // API 密鑰（如果使用自訂模型）  
  "cursor.apiKey": "your-api-key-here",  
  
  // 模型溫度（創意度 0-1）  
  "cursor.temperature": 0.7,  
  
  // 最大標記數  
  "cursor.maxTokens": 4096,  
  
  // 上下文大小  
  "cursor.contextSize": "extended"  
}
```

2. Chat 模式配置

多模式支援

```
{
  // Chat 快捷鍵模式
  "cursor.chat.openMode": "sidebar", // 或 "panel" / "dialog"

  // 記憶對話歷史
  "cursor.chat.rememberHistory": true,
  "cursor.chat.historyLength": 10,

  // 代碼完成模式
  "cursor.chat.defaultLanguage": "javascript",

  // 自動代碼高亮
  "cursor.chat.syntaxHighlight": true
}
```

3. Tab 自動補全高級配置

```
{
  // 補全觸發器
  "cursor.completion.triggerCharacters": [".", "{", "("],

  // 過濾重複建議
  "cursor.completion.filterDuplicates": true,

  // 最小補全字符
  "cursor.completion.minCharacters": 1,

  // 自動接受補全
  "cursor.completion.autoAccept": false,

  // 補全超時（毫秒）
  "cursor.completion.timeout": 5000
}
```

進階 Chat 技巧

上下文管理

1. 檔案引用

在 Chat 中使用 @filename 引用特定檔案
@src/components/Button.tsx - 引用整個檔案

使用 `@symbol:functionName` - 引用特定函數
`@symbol:calculateTotal`

2. 符號引用

快速找到並引用：
`@symbol:MyClass` - 引用類別
`@symbol:myFunction` - 引用函數
`@symbol:myVariable` - 引用變數

3. 資料夾引用

`@folder:src/utls` - 引用資料夾中的所有檔案

限制上下文大小以提高速度

專業提示詞 (Advanced Prompts)

重構範本

我想把這個函數重構為更函數式的風格，
並添加類型註解和錯誤處理。
保留原有的功能但改進可讀性。

性能優化

分析這段代碼的性能瓶頸。
建議使用 Big O 表示法的複雜度分析，
並提出具體的優化方案。

安全審計

從安全角度檢查這段代碼。
尋找 SQL 注入、XSS、CSRF 等潛在漏洞。
提供修復建議和最佳實踐。

文檔生成

為這個 API 端點生成完整的 OpenAPI/Swagger 文檔。
包括請求/響應示例和所有可能的錯誤碼。

高級開發工作流

工作區管理

1. 多資料夾工作區

建立 `.code-workspace` 檔案：

```
{
  "folders": [
    {
      "path": "frontend",
      "name": "Frontend"
    },
    {
      "path": "backend",
      "name": "Backend"
    },
    {
      "path": "shared",
      "name": "Shared Libraries"
    }
  ],
  "settings": {
    "typescript.tsdk": "node_modules/typescript/lib",
    "editor.formatOnSave": true,
    "editor.tabSize": 2
  }
}
```

開啟工作區：

File → Open Workspace from File → 選擇 `.code-workspace` 檔案

2. 工作區特定快捷鍵

在 `.vscode/keybindings.json` 中設定：

```
[
  {
    "key": "ctrl+shift+f10",
```

```
"command": "workbench.action.switchWindow"
},
{
  "key": "ctrl+alt+1",
  "command": "workbench.action.focusFirstEditorGroup"
},
{
  "key": "ctrl+alt+2",
  "command": "workbench.action.focusSecondEditorGroup"
}
]
```

進階除錯設定

1. 除錯配置 (launch.json)

建立 `.vscode/launch.json` :

Node.js 除錯

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "type": "node",
      "request": "launch",
      "name": "Launch Node Program",
      "program": "${workspaceFolder}/app.js",
      "skipFiles": [
        "<node_internals>/**",
        "${workspaceFolder}/node_modules/**"
      ],
      "console": "integratedTerminal",
      "internalConsoleOptions": "neverOpen",
      "outFiles": [
        "${workspaceFolder}/dist/**/*.js"
      ]
    }
  ]
}
```

Python 除錯

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Python: Current File",
```

```

    "type": "python",
    "request": "launch",
    "program": "${file}",
    "console": "integratedTerminal",
    "justMyCode": true,
    "args": []
  },
  {
    "name": "Python: Flask",
    "type": "python",
    "request": "launch",
    "module": "flask",
    "env": {
      "FLASK_APP": "app.py",
      "FLASK_ENV": "development"
    },
    "args": ["run"],
    "jinja": true
  }
]
}

```

Web 除錯 (Chrome)

```

{
  "version": "0.2.0",
  "configurations": [
    {
      "type": "chrome",
      "request": "launch",
      "name": "Launch Chrome",
      "url": "http://localhost:3000",
      "webRoot": "${workspaceFolder}",
      "runtimeArgs": ["--remote-debugging-port=9222"]
    }
  ]
}

```

2. 條件斷點和日誌點

右鍵點擊行號 → 「Add Conditional Breakpoint」
輸入條件，例如：count > 10

或設定「Logpoint」來打印變數而不暫停執行

任務自動化 (tasks.json)

建立 `.vscode/tasks.json`：

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "Build TypeScript",
      "type": "shell",
      "command": "tsc",
      "args": ["--project", "tsconfig.json"],
      "problemMatcher": ["$tsc"],
      "group": {
        "kind": "build",
        "isDefault": true
      }
    },
    {
      "label": "Run Tests",
      "type": "shell",
      "command": "npm",
      "args": ["test"],
      "problemMatcher": [],
      "group": {
        "kind": "test",
        "isDefault": true
      }
    },
    {
      "label": "Format Code",
      "type": "shell",
      "command": "prettier",
      "args": ["--write", "."],
      "runOptions": {
        "runOn": "folderOpen"
      }
    },
    {
      "label": "Start Dev Server",
      "type": "shell",
      "command": "npm",
      "args": ["run", "dev"],
      "isBackground": true,
      "problemMatcher": {
        "pattern": {
          "regexp": "^.*$",
          "line": 0
        },
        "background": {
          "activeOnStart": true,
          "watching": "start"
        }
      }
    }
  ]
}
```

```
]
}
```

運行任務：

按 Ctrl + Shift + B（構建）
或 Ctrl + Shift + P → "Run Task"

性能優化與調試

編輯器性能優化

1. 大檔案處理

```
{
  // 禁用大檔案的某些功能
  "editor.largeFileOptimizations": true,

  // 大檔案大小閾值 (KB)
  "editor.largeFileOptimizations": true,

  // 禁用代碼摺疊
  "[plaintext]": {
    "editor.folding": false,
    "editor.wordWrap": "off"
  }
}
```

2. 擴展性能監控

在命令面板中搜尋 "Developer: Show Running Extensions"
查看哪些擴展佔用最多資源

3. 搜尋和 IntelliSense 優化

```
{
  // 限制搜尋結果
  "search.maxResults": 10000,

  // 禁用某些檔案類型的 IntelliSense
  "files.watcherExclude": {
    "**/.git/objects/**": true,
    "**/.git/subtree-cache/**": true,
  }
}
```

```
    "**/node_modules/**": true,  
    "**/.venv/**": true  
  },  
  
  // 搜尋延遲  
  "search.searchOnType": true,  
  "search.searchOnTypeDebouncePeriod": 300  
}
```

內存和 CPU 使用分析

開啟開發者工具

Ctrl + Shift + P → "Developer: Open Process Explorer"

或者

Ctrl + Shift + P → "Developer: Show All Commands"

找到性能相關命令

監控日誌

Ctrl + Shift + P → "Developer: Toggle Developer Tools"

查看控制台、網路、性能等信息

進階版本控制

Git 高級功能

1. 互動式 Git 歷史

在 Source Control 面板中：

- 檢視提交圖表
- 比較分支
- 檢視代碼責任 (Blame)

2. Git 命令快捷鍵

```
[  
  {  
    "key": "ctrl+shift+g c",  
    "command": "git.commitStaged",  
    "when": "gitBaseResource == 'file'"  
  }  
]
```

```
    },  
    {  
      "key": "ctrl+shift+g p",  
      "command": "git.push"  
    },  
    {  
      "key": "ctrl+shift+g l",  
      "command": "git.openFile",  
      "args": ["HEAD~1"]  
    }  
  ]
```

3. Git 鉤子自動化

在專案根目錄建立 `.husky` 目錄用於 Git 鉤子：

```
# 安裝 husky  
npm install husky --save-dev  
npx husky install  
  
# 建立預提交鉤子  
npx husky add .husky/pre-commit "npm run format && npm run lint"  
  
# 建立提交訊息鉤子  
npx husky add .husky/commit-msg "npm run validate-commit"
```

4. 高級 Git 配置

```
{  
  "git.autofetch": true,  
  "git.autoFetchPeriod": 120,  
  "git.fetchOnPull": true,  
  "git.pullBeforeCheckout": true,  
  "git.mergeEditor": true,  
  "git.ignoreMissingGitWarning": false  
}
```

分支管理策略

Git Flow 工作流

```
# 建立特性分支  
git checkout -b feature/new-feature  
  
# 完成特性
```

```
git checkout develop
git merge feature/new-feature
git push origin develop

# 發佈版本
git checkout -b release/v1.0.0 develop
# 修正版本號
git checkout main
git merge release/v1.0.0
```

配置默認分支策略

在 `.vscode/settings.json` :

```
{
  "git.defaultCloneDirectory": "${userHome}/projects",
  "git.branchProtection": true,
  "git.branchWhitelist": ["main", "develop"]
}
```

團隊協作功能

Live Share 設定

1. 安裝和配置

Ctrl + Shift + X → 搜尋 "Live Share" → 安裝微軟官方擴展

2. 開始共享會話

Ctrl + Shift + P → "Live Share: Start collaboration session"

分享產生的連結給團隊成員

3. 高級設定

```
{
  "liveshare.presence": true,
  "liveshare.languageServices": true,
  "liveshare.autoJoinPrimarySession": false,
  "liveshare.shareCursorPosition": true,
  "liveshare.shareTerminal": true,
```



```
"liveshare.showModalNotifications": true  
}
```

評論和註解系統

建立任務評論

按 **Ctrl + Shift + /** 建立評論：

```
// TODO: 實現用戶認證  
// FIXME: 修復邊界情況  
// HACK: 臨時解決方案，需要重構  
// NOTE: 重要的設計決定  
// XXX: 警告，可能有問題
```

使用 Todo Tree 擴展跟蹤所有任務：

Ctrl + Shift + X → 安裝 "Todo Tree"

自動化與擴展開發

建立自訂擴展

1. 基礎結構

```
# 建立擴展框架  
npm install -g yo generator-code  
yo code  
# 選擇 TypeScript  
# 輸入擴展名稱
```

2. Hello World 擴展

在 `src/extension.ts`：

```
import * as vscode from 'vscode';  
  
export function activate(context: vscode.ExtensionContext) {  
  let disposable = vscode.commands.registerCommand(  
    'myextension.helloWorld',  
    () => {  
      vscode.window.showInformationMessage('Hello from My Extension!');  
    }  
  );  
}
```

```
    }  
  );  
  
  context.subscriptions.push(disposable);  
}  
  
export function deactivate() {}
```

3. 打包和發佈

```
# 安裝 vsce  
npm install -g vsce  
  
# 打包擴展  
vsce package  
  
# 發佈到市場  
vsce publish
```

4. 常用 API

```
// 命令註冊  
vscode.commands.registerCommand('myext.command', () => {});  
  
// 快捷鍵綁定  
// 在 package.json 中設定  
  
// 狀態欄項目  
const statusBar = vscode.window.createStatusBarItem();  
statusBar.text = "My Status";  
statusBar.show();  
  
// 檔案系統監視  
const watcher = vscode.workspace.createFileSystemWatcher(  
  new vscode.RelativePattern(  
    vscode.workspace.workspaceFolders?.[0],  
    '**/*.ts'  
  )  
);  
  
watcher.onDidCreate(uri => {  
  console.log('File created:', uri);  
});
```

1. 檔案自動轉換

建立任務自動轉換檔案格式：

```
{
  "label": "Auto Convert Markdown to HTML",
  "type": "shell",
  "command": "pandoc",
  "args": [
    "${file}",
    "-t",
    "html",
    "-o",
    "${fileBasenameNoExtension}.html"
  ],
  "runOptions": {
    "runOn": "folderOpen"
  }
}
```

2. 代碼生成自動化

使用 snippets 和 Cursor AI：

建立 `.vscode/vue.code-snippets`：

```
{
  "Vue Component": {
    "prefix": "vue",
    "body": [
      "<template>",
      "  <div class=\"${1:component}\">",
      "    $0",
      "  </div>",
      "</template>",
      "",
      "<script setup>",
      "import { ref } from 'vue'",
      "",
      "defineProps({})",
      "const state = ref(null)",
      "</script>",
      "",
      "<style scoped>",
      ".$1 {",
      "  /* styles */",
      "}",
      "</style>"
    ]
  }
}
```

```
}  
}
```

3. 提交前自動化

在 `package.json`：

```
{  
  "scripts": {  
    "precommit": "lint-staged && npm run test",  
    "prepare": "husky install"  
  },  
  "lint-staged": {  
    "*.ts": ["eslint --fix", "prettier --write"],  
    "*.vue": ["eslint --fix", "prettier --write"],  
    "*.json": ["prettier --write"]  
  }  
}
```

安全與隱私設定

代碼隱私

1. 排除敏感檔案

在 `.gitignore` 和 `settings.json`：

```
{  
  "files.exclude": {  
    "**/.env": true,  
    "**/.env.local": true,  
    "**/*.key": true,  
    "**/*.pem": true,  
    "**/secrets/**": true  
  },  
  "search.exclude": {  
    "**/.env*": true,  
    "**/secrets": true  
  }  
}
```

2. AI 隱私設定

```
{  
  // 不發送敏感資訊到 AI
```

```
"cursor.privacy.enableTelemetry": false,  
"cursor.privacy.excludePatterns": [  
  "**/.env*",  
  "**/*password*",  
  "**/*secret*",  
  "**/*token*"  
],  
  
// 本地 AI 模式 (如支援)  
"cursor.aiMode": "local"  
}
```

3. 認證令牌管理

使用環境變數而非硬編碼：

```
# .env 檔案  
API_KEY=your_key_here  
DATABASE_URL=your_db_url  
  
# 在程式中使用  
const apiKey = process.env.API_KEY;
```

依賴安全掃描

1. 安裝安全工具

```
npm install -g snyk  
  
# 掃描依賴漏洞  
snyk test  
  
# 持續監控  
snyk monitor
```

2. 自動化安全檢查

在 `.vscode/tasks.json`：

```
{  
  "label": "Security Audit",  
  "type": "shell",  
  "command": "npm",  
  "args": ["audit"],  
  "runOptions": {
```

```
    "runOn": "folderOpen"
  }
}
```

進階快捷鍵自訂

自訂快捷鍵完整指南

編輯 `.vscode/keybindings.json`：

```
[
  // AI 快捷鍵
  {
    "key": "ctrl+shift+a",
    "command": "cursor.chat",
    "when": "editorTextFocus"
  },
  {
    "key": "ctrl+i",
    "command": "cursor.refactor",
    "when": "editorTextFocus"
  },

  // 編輯快捷鍵
  {
    "key": "ctrl+alt+m",
    "command": "editor.action.showContextMenu",
    "when": "editorTextFocus"
  },
  {
    "key": "ctrl+alt+e",
    "command": "editor.action.expandSelectionByLine",
    "when": "editorTextFocus"
  },

  // 導航快捷鍵
  {
    "key": "ctrl+l",
    "command": "workbench.action.quickOpen",
    "when": "!inputFocus"
  },
  {
    "key": "alt+left",
    "command": "workbench.action.navigateBack"
  },
  {
    "key": "alt+right",
    "command": "workbench.action.navigateForward"
  },
]
```

```
// Git 快捷鍵
{
  "key": "ctrl+shift+g g",
  "command": "git.openRepository"
},
{
  "key": "ctrl+shift+g d",
  "command": "git.viewFileHistory"
},

// 測試快捷鍵
{
  "key": "ctrl+shift+t",
  "command": "testing.runAllTests"
},
{
  "key": "ctrl+alt+t",
  "command": "testing.runAtCursor"
}
]
```

故障排查與優化



常見問題診斷

1. 性能問題診斷

Ctrl + Shift + P → "Developer: Performance"
查看時間軸和性能指標

2. 擴展衝突檢查

Ctrl + Shift + P → "Developer: Show Running Extensions"
逐個禁用擴展找出衝突源

3. 清除快取

```
# Windows
rmdir %APPDATA%\Cursor\User\workspaceCache

# Mac
rm -rf ~/Library/Application\ Support/Cursor/User/workspaceCache
```

```
# Linux  
rm -rf ~/.config/Cursor/User/workspaceCache
```

☞ 速度優化清單

前 10 個優化

1. 禁用不必要的擴展

在擴展面板中禁用工作區不需要的擴展

2. 增加 AI 補全延遲

```
"editor.quickSuggestionsDelay": 50
```

3. 限制搜尋結果

```
"search.maxResults": 5000
```

4. 禁用檔案監視

```
"files.watcherExclude": {  
  "**/node_modules/**": true,  
  "**/.git/**": true  
}
```

5. 使用工作區設定

針對特定專案優化設定

6. 關閉不必要的預覽

```
"editor.preview": false
```

7. 禁用不必要的格式化


```
"editor.formatOnSave": false
```

8. 增加終端滾動回溯

```
"terminal.integrated.scrollback": 1000
```

9. 關閉遠端容器同步

```
"remote.autoForwardPorts": false
```

10. 重啟編輯器

```
Ctrl + Shift + P → "Reload Window"
```

進階調試日誌

啟用詳細日誌

在命令行啟動 Cursor：

```
# Windows
code --log debug

# Mac/Linux
code --log debug
```

查看日誌

```
Ctrl + Shift + P → "Output" → 選擇不同的頻道查看日誌
```

進階工作流範例

範例 1：全棧開發工作流

.code-workspace 配置

```
{
  "folders": [
    {"path": "frontend"},
    {"path": "backend"},
    {"path": "database"}
  ],
  "settings": {
    "editor.formatOnSave": true,
    "editor.defaultFormatter": "esbenp.prettier-vscode"
  },
  "extensions": {
    "recommendations": [
      "ms-vscode.vscode-typescript-next",
      "prismaticai.prisma",
      "ms-python.python",
      "eamodio.gitlens",
      "ms-vscode.makefile-tools"
    ]
  }
}
```

任務配置

```
{
  "tasks": [
    {
      "label": "Full Build",
      "dependsOn": ["Build Backend", "Build Frontend"],
      "group": {"kind": "build", "isDefault": true}
    },
    {
      "label": "Start All",
      "dependsOn": ["Start Backend", "Start Frontend"],
      "runSequence": ["Start Backend", "Start Frontend"]
    }
  ]
}
```

範例 2：機器學習開發

Python 專項設定

```
{
  "[python]": {
    "editor.defaultFormatter": "ms-python.python",
    "editor.formatOnSave": true,
    "editor.rulers": [79, 99],
  }
}
```

```
"editor.tabSize": 4
},
"python.linting.enabled": true,
"python.linting.pylintEnabled": true,
"python.linting.mypyEnabled": true,
"python.formatting.provider": "black"
}
```

Jupyter Notebook 整合

```
{
  "notebook.kernelShutdownTimeout": 10000,
  "notebook.formatOnSave.enabled": true,
  "jupyter.askForKernelRestart": true,
  "python.experiments.enabled": true
}
```

範例 3：微服務開發

Docker 整合

```
# 安裝 Docker 擴展
Ctrl + Shift + X → "Docker"

# 任務配置
{
  "label": "Docker Build All",
  "type": "shell",
  "command": "docker-compose",
  "args": ["build"]
}
```

Kubernetes 支援

```
# 安裝 Kubernetes 擴展
Ctrl + Shift + X → "Kubernetes"

# 快速部署快捷鍵
{
  "key": "ctrl+shift+k",
  "command": "extension.vsKubernetesApply"
}
```

官方文檔

- **Cursor 完整文檔**：<https://cursor.so/docs>
- **設定參考**：<https://cursor.so/docs/settings>
- **快捷鍵參考**：<https://cursor.so/docs/keybindings>

社群資源

- **GitHub Discussions**：報告問題和分享想法
- **Discord 社群**：即時支援和討論
- **YouTube 頻道**：進階教學和技巧

推薦擴展（進階）

開發工具

- GitLens - 進階 Git 功能
- REST Client - 測試 API
- Docker - 容器管理
- Kubernetes - K8s 管理

代碼質量

- ESLint - JavaScript 檢查
- Pylint - Python 檢查
- SonarLint - 代碼質量
- CodeMetrics - 複雜度分析

生產力

- Tabnine - AI 程式碼補全
- GitHub Copilot - AI 助手
- Peacock - 工作區著色
- Project Manager - 專案管理



最佳實踐檢查表

設定優化

- ☐ 根據項目類型自訂 `settings.json`
- ☐ 建立工作區特定設定
- ☐ 配置格式化器和 linter
- ☐ 設定 Git 鉤子自動化

性能優化

- ☐ 禁用不必要的擴展
- ☐ 配置檔案監視排除
- ☐ 調整 AI 補全延遲
- ☐ 啟用大檔案優化

安全設定

- ☐ 排除敏感檔案
- ☐ 配置 AI 隱私設定
- ☐ 設定依賴安全掃描
- ☐ 管理認證令牌

工作流優化

- ☐ 建立自訂快捷鍵
- ☐ 配置任務自動化
- ☐ 設定除錯配置
- ☐ 建立代碼片段範本

結論

掌握 Cursor IDE 的進階功能可以：

- ☒ 大幅提升開發效率（30-50%）
- ☒ 改善代碼品質
- ☒ 建立可重複的工作流
- ☒ 支援複雜的開發場景
- ☒ 實現團隊協作

定期查閱本手冊並實踐各種功能，您將成為 Cursor IDE 的真正高手！

最後更新：2024年7月

手冊版本：2.0 (進階版)

適用版本：Cursor 最新版本

難度等級：★★★★★ (進階)